Design Patterns

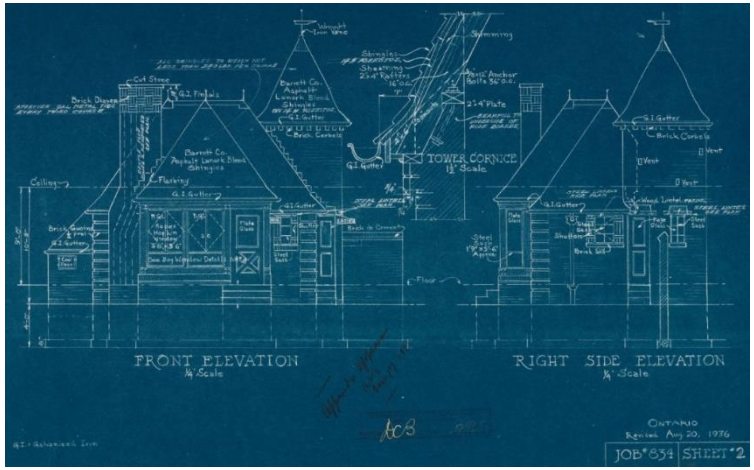
Design Pattern

A blueprint or schematic to solve a re-occurring problem.

- i.e. a common battle-tested solution to a frequently occurring problem.

Each pattern has a **name**, an **intent**, and **solves a specific problem**.

- We can use design patterns to communicate our thoughts!
- Unfortunately this is not always an exact science.



Design Patterns

Creational Patterns:

- Abstract Factory
- **Builder**
- Factory Method
- Singleton

Structural Patterns:

- Adapter
- Composite
- **Decorator**
- Proxy

Behavioral Patterns:

- Command
- Iterator
- Memento
- Null Object
- **Observer**
- State
- **Strategy**
- Visitor

And many many more…

STRATEGY Pattern

Problem:

- We have a class that should vary its behavior depending on circumstance, but the class should not be responsible for the concrete behavior itself.

Solution:

- Abstract the concrete behavior into a STRATEGY interface.
- Augment the class to take an instance of the STRATEGY.
- Implement the varying behavior by delegation to the STRATEGY object.

```
public enum AccountType {
    CHECKING,
    SAVING
}
```

STRATEGY: Motivation

```
public class BankAccount {
    private double balance;
    private AccountType type;
    public BankAccount(double balance, AccountType type) {
        this.balance = balance;
        this.type = type;
    }
    public double getBalance() { ... }
    public void deposit(double amount) { ... }
    public void withdraw(double amount) { ... }
    public void accrueInterest() {
        switch (type) {
            case CHECKING:
                this.balance = this.balance * 1.000001;
                break;
            case SAVING:
                this.balance = this.balance * 1.05;
                break;
        }
    }
}
```

STRATEGY: Interface & Instances

```
public interface InterestStrategy {
    double getInterest(double balance);
}
```

```
public class CheckingAccountInterest implements InterestStrategy {
    public double getInterest(double balance) {
        return balance * 0.000001;
    }
}
```

```
public class SavingsAccountInterest implements InterestStrategy {
    public double getInterest(double balance) {
        return balance * 0.05;
    }
}
```

STRATEGY: Delegate Behavior

```
public class BankAccount {

    private double balance;
    private InterestStrategy interestStrategy;

    public BankAccount(double balance,
                       InterestStrategy interestStrategy) {
        this.balance = balance;
        this.interestStrategy = interestStrategy;
    }

    public double getBalance() { ... }
    public void deposit(double amount) { ... }
    public void withdraw(double amount) { ... }
    public void accrueInterest() {
        this.balance = this.balance +
                       interestStrategy.getInterest(this.balance);
    }
}
```

Observer Pattern

Problem:

- We want to be notified whenever changes happen to a (subject) object.
- For example,
  - Whenever a button is pressed, we want to execute some code.
  - Whenever a key is pressed, we want to execute some code.
  - Whenever a network request completes, we want to execute some code.

Solution:

- We introduce an interface for listeners.
  - i.e. an interface for the methods that should be executed when the event happens.
- We allow clients (other objects) to register listeners on the subject object.
- Whenever an event happens to the subject object, it invokes the listeners.

OBSERVER: Listener Interface

```
public interface AccountListener {

    void notify(double newBalance, double oldBalance);

}
```

OBSERVER: Subject Object

```
public class BankAccount {
    private double balance;
    private List<AccountListener> listeners = new ArrayList();

    public void addListener(AccountListener listener){
        listeners.add(listener);
    }
    public void deposit(double amount) {
        double oldBalance = this.balance;
        this.balance = this.balance + amount;
        for(AccountListener listener: this.listeners){
            listener.notify(this.balance, oldBalance);
        }
    }
}
```

OBSERVER: Listener Object

```
public class BankStatistics implements AccountListener{
    double totalBankBalance;

    public BankStatistics(double totalBankBalance) {
        this.totalBankBalance = totalBankBalance;
    }

    @Override
    public void notify(double newBalance, double oldBalance) {
        double difference = newBalance-oldBalance;
        totalBankBalance+=difference;
    }
}
```

DECORATOR Pattern

Problem:

- We want to augment an existing class with additional behavior.
- For example,
  - To perform logging of some operations performed on the class.
  - To enforce additional security measures.
  - To cache certain computations (e.g. `BufferedReader`)

Solution:

- We refactor the API of the class to an interface.
- We let the original class implement the interface.
- We implement a DECORATOR class which implements the interface.
  - The class contains a reference to an instance of the original class.
  - The class forwards all methods to the original instance.
  - When desired, the class may intercept certain methods and perform additional computation.

DECORATOR: Existing Class

```
public class BankAccount {

    public String getOwner() { ... }

    public double getBalance() { ... }

    public void deposit(double amount) { ... }

    public void withdraw(double amount) { ... }

    public void accrueInterest() { ... }

    public void transferTo(BankAccount otherAccount) { ... }

}
```

DECORATOR: Turn into Interface

```
public interface BankAccount {

    public String getOwner();

    public double getBalance();

    public void deposit(double amount);

    public void withdraw(double amount);

    public void accrueInterest();

    public void transferTo(BankAccount otherAccount);

}
```

DECORATOR: The Decorator

```
public class BankAccountDecorator implements BankAccount {
    private BankAccount account;

    public BankAccountDecorator(BankAccount account) {
        this.account = account;
    }

    public String getOwner() { return account.getOwner(); }
    public double getBalance() { return account.getBalance(); }
    public void deposit(double amount) {
        System.out.println("Deposit: " + amount);
        account.deposit(amount);
    }

    public void transferTo(BankAccount otherAccount) {
        if (account.getOwner().equals(otherAccount.getOwner())) {
            account.transferTo(otherAccount);
        }
    }
}
```

BUILDER Pattern

Problem:

- We want to construct a **complex object**.
- The class of the complex object has many fields.
- Some fields are optional with default values, some are not.
- Some fields have complex invariants, some do not.
- Using a single constructor (or a multitude of constructors) is verbose.

Solution:

- We introduce a **Builder** object to aid construction of the complex object.
- The Builder allows the complex object to be constructed *piece-by-piece*.

BUILDER: The Complex Class

```
public class BankAccount {
    private String firstName;
    private String middleName;
    private String lastName;
    private String socialSecurity;
    private String streetAddress;
    private String streetNumber;
    private String city;
    private String country;
    private double balance;
    private boolean allowOverdraft;
    private double borrowRate;
    private double interestRate;
    private Date accountCreated;
    private Date accountChanged;
    private Date accountArchived;
    private List<Transaction> transactions;
    // ...
}
```

BUILDER: The Builder Class

```
public class BankAccountBuilder {
    private String firstName;
    private String middleName;
    private String lastName;
    public BankAccountBuilder setFirstName(String firstName) {
        this.firstName = firstName;
        return this;
    }
    public BankAccountBuilder setMiddleName(String middleName) {
        this.middleName = middleName;
        return this;
    }
    public BankAccountBuilder setLastName(String lastName) {
        this.lastName = lastName;
        return this;
    }
    public BankAccount build() {
        return new BankAccount(...);
    }
}
```

BUILDER: Using It

```
public static void main(String[] args) {

    BankAccountBuilder builder = new BankAccountBuilder();
    builder.setFirstName("Magnus")
           .setLastName("Madsen");
    BankAccount account = builder.build();

}
```

More Design Patterns

